

Data Lab 1 实验报告

整数与浮点数位级运算的实现、验证与操作数优化

姓名	梁宇华
学号	25301020133

一、实验目的

1. 掌握整数的补码表示以及移位、按位逻辑运算在位级程序设计中的应用。
2. 通过位级运算实现整数、区间、饱和运算以及奇偶校验、位计数、位反转等函数。
3. 理解 IEEE 754 单精度浮点数的符号位、指数域和尾数字段，并完成浮点数的缩放、舍入及整数转换。
4. 使用 `btest` 验证函数的功能正确性，并使用操作数检查程序检查 Data Lab 对每个函数的最大操作数限制。

二、实验要求与编码规则

本实验的 `bits.c` 文件规定，整数部分只能使用 0~255 的整数常量以及规定的位级运算；禁止使用 `if`、`for`、`while` 等控制结构、函数调用、宏、类型转换及其他数据类型。机器模型采用 32 位二进制补码，并假定右移为算术右移。

浮点部分的规则相对宽松，可以使用 `int/unsigned`、条件语句和循环，也允许使用任意整数常量，但仍不能使用浮点数据类型、浮点运算或额外函数。

题目同时规定了每个函数的最大操作数。`assignment '='` 不计入操作数，因此可以通过增加局部变量来组织表达式，但真正参与计算的运算符数量必须满足上限。

三、实验内容与实现思路

P1 `signMask`

利用 1 左移 31 位直接构造最高位为 1、其余位为 0 的掩码。

P2 `bitXor`

利用德摩根定律和与、取反运算，在不直接使用 `^` 的情况下构造异或。

P3 `negativePart`

利用 `x` 的符号扩展得到全 0 或全 1 掩码；当 `x` 为负数时用补码取负，否则返回 0。

P4 `copyByteWithin`

先根据 `src`、`dst` 计算字节移位量，取出源字节，再清除目标字节并把源字节移入目标位置。

P5 `logicalShift`

利用算术右移得到初步结果，再构造掩码清除符号扩展产生的高位 1，从而模拟逻辑右移。

P6 swapNibblePairs

构造每个字节低 4 位的掩码，同时提取高、低半字节并分别左移、右移 4 位后合并。

P7 secondLowestZeroBit

先对 x 取反，把原来的 0 变成 1；使用 $x \& (\sim x + 1)$ 的低位提取技巧依次取得最低和第二低的 1 位。

P8 oddParity

通过 16、8、4、2、1 位的逐级异或折叠，把 32 位信息压缩到最低位，再取反得到题目定义的奇偶校验结果。

P9 rotateRightBits

把右移得到的低位部分与左移得到的回卷部分进行或运算，并利用掩码避免算术右移的符号扩展。

P10 roundEvenPow2

分离 x 除以 2^n 后的整数部分和余数；比较余数与半个步长，并在恰好一半时根据商的最低位实现 ties-to-even。

P11 midpointTowardFirst

通过 x 与 y 的符号关系判断大小规律，利用无溢出的平均值形式 $(x \& y) + ((x \oplus y) \gg 1)$ ，再处理奇数差值的取整方向。

P12 isBetweenEitherOrder

分别判断 x 相对于 a 、 b 的大小关系，并考虑不同符号时减法可能溢出的情况，最后组合出两个端点顺序下的区间判断。

P13 mul5Sat

先计算 $x*4$ 和 $x*5$ ，并分别检测中间乘法和最终加法的溢出；发生溢出时根据原数符号选择 `INT_MAX` 或 `INT_MIN`。

P14 classifyAdd3

将 $x+y$ 与再加 z 分成两个阶段检测有符号溢出，并记录溢出的方向，最后合并得到 1、-1 或 0。

P15 floatScaleThreeHalves

拆分符号、指数和尾数；分别处理 NaN/无穷、非规格化数和规格化数。使用整数形式实现乘 $3/2$ ，并对舍入和规格化边界进行处理。

P16 floatRoundEven

根据指数判断数值范围；对绝对值小于 0.5 的情况返回带符号零，对 0.5 附近采用偶数舍入；对于一般情况提取整数部分、guard 位和 sticky 位完成舍入。

P17 float_i2f

确定符号和最高有效位位置，将整数规格化到 23 位尾数范围，并依据被舍弃部分执行 round-to-nearest-even。

P18 bitCount

利用 1、2、4、8、16 位分组掩码逐级累加每组中的 1 的数量，最终得到 32 位整数的置位位数。

P19 bitReverse

逐级交换 16、8、4、2、1 位分组，实现 32 位整数的完全位反转；同时构造掩码时遵守整数常量不超过 0xFF 的限制。

四、正确性验证

本次实验使用 btest 对全部函数进行测试。此前调试过程中发现并修正了 midpointTowardFirst、floatScaleThreeHalves、floatRoundEven、bitCount 和 bitReverse 的边界问题；在完成修正后，用户终端测试结果显示 btest 已无错误。

典型边界问题包括：

- midpointTowardFirst(0, 0x80000001) 的数学中点为 -1073741823.5，按题意应选择更靠近第一个参数 0 的整数 -1073741823。
- floatScaleThreeHalves 的规格化过程中需要考虑右移产生的被舍弃位，否则会出现最后一位少 1 的舍入误差。
- floatRoundEven 对 E=-1 的情况不能简单全部返回 0；略大于 0.5 的值应舍入为 1.0，同时保留负零规则。
- bitCount 必须正确处理 0x80000000 这样的单个最高位为 1 的输入。
- bitReverse 既要保证位反转正确，又必须满足 Data Lab 对大整数常量的限制。

五、操作数检查与优化

最终阶段使用 ./check_ops.py bits.c 检查操作数。一次检查结果显示，主要函数的操作数均在规定上限以内，其中 roundEvenPow2 为 22/24，midpointTowardFirst 为 22/32，isBetweenEitherOrder 为 48/48，mul5Sat 为 21/30，classifyAdd3 为 33/52，floatScaleThreeHalves 为 45/60，floatRoundEven 为 50/65，float_i2f 为 26/40，bitCount 为 36/40；logicalShift、swapNibblePairs、secondLowestZeroBit、oddParity 和 rotateRightBits 也均低于对应上限。

bitReverse 是操作数优化中的重点。直接使用 0x55555555、0x33333333 等大常量虽然可以降低表达式长度，但违反整数 Data Lab 的“常量只能为 0~0xFF”规则。因此最终版本通过 0xFF 逐级构造掩码，再完成 16、8、4、2、1 位的交换，在满足合法常量限制的同时控制在题目规定的 34 个操作以内。

六、实验结果

检查项目	结果	说明
btest 功能测试	通过	最终测试无错误
整数编码规则	通过	使用规定的位级运算与常量范围
浮点编码规则	通过	使用 int/unsigned 和条件控制，未使用浮点运算
操作数检查	通过	各函数均满足对应最大操作数要求
bitReverse	通过	功能正确且满足 34 操作上限

图 1 运行 check_ops.py bits.c 结果

```
all checks passed: 110/110
● Jackson706@localhost:~/work2$ ./check_ops.py bits.c
signMask: 1/2 operators
bitXor: 8/8 operators
negativePart: 4/6 operators
copyByteWithin: 9/12 operators
logicalShift: 6/20 operators
swapNibblePairs: 9/18 operators
secondLowestZeroBit: 8/8 operators
oddParity: 12/56 operators
rotateRightBits: 12/16 operators
roundEvenPow2: 22/24 operators
midpointTowardFirst: 22/32 operators
isBetweenEitherOrder: 48/48 operators
mul5Sat: 21/30 operators
classifyAdd3: 33/52 operators
floatScaleThreeHalves: 45/60 operators
floatRoundEven: 50/65 operators
float_i2f: 26/40 operators
bitCount: 36/40 operators
bitReverse: 34/34 operators
All 19 functions passed operator checks.
```

图 2 运行 ./btest 结果

```
● Jackson706@localhost:~/work2$ ./btest
Score  Rating  Errors  Function
1      1      0      signMask
2      2      0      bitXor
3      3      0      negativePart
4      4      0      copyByteWithin
4      4      0      logicalShift
4      4      0      swapNibblePairs
4      4      0      secondLowestZeroBit
5      5      0      oddParity
5      5      0      rotateRightBits
5      5      0      roundEvenPow2
5      5      0      midpointTowardFirst
7      7      0      isBetweenEitherOrder
7      7      0      mul5Sat
7      7      0      classifyAdd3
7      7      0      floatScaleThreeHalves
10     10     0      floatRoundEven
10     10     0      float_i2f
10     10     0      bitCount
10     10     0      bitReverse
Total points: 110/110
```

七、实验总结

通过本次 Data Lab 实验，我进一步理解了整数在 32 位补码机器模型下的位级表示，并掌握了利用移位、按位逻辑运算和补码性质实现复杂整数运算的方法。实验中最困难的部分是边界情况：例如有符号溢出、负数算术右移、舍入到偶数、非规格化浮点数以及 IEEE 754 指数溢出。

调试过程中可以看到，功能正确并不意味着程序已经符合 Data Lab 要求。bitReverse 的例子尤其明显：使用 0x55555555 等常量可以得到简洁实现，但会被 dlc 判定为非法；因此需要同时考虑语义正确性、编码规则和操作数上限。最终通过 btest 与操作数检查相结合，可以较完整地验证提交代码。

本实验也说明了位级程序设计中“先分析数据表示，再设计掩码和移位关系”的重要性。对于浮点题，则需要先按照 IEEE 754 的字段拆分问题，再分别处理规格化数、非规格化数、零、无穷和 NaN 等特殊情况。

八、参考资料

1. Bryant, O'Hallaron, Computer Systems: A Programmer's Perspective (CS:APP).
2. 本课程 Data Lab 实验说明及 bits.c 文件头部编码规则。
3. 本实验提供的 dlc、btest 与操作数检查脚本。